Creating Instruction Fine-Tuning (IFT) Data for Athena-CTI-LLM

MITRE ATT&CK

1. Overview of MITRE ATT&CK Dataset

MITRE ATT&CK is a structured knowledge base detailing real-world cybersecurity attack techniques and methods used by threat actors. It provides:

- **Tactics** (attacker goals, e.g., Initial Access, Persistence)
- **Techniques** (specific methods, e.g., Phishing, Spear-phishing Attachment)
- **Detailed Descriptions** (clear explanations of each technique, often in markdown format)
- **Detection and Mitigation Advice** (recommendations for defending and detecting threats)
- **Relationships** (associations between threat actors, malware, and techniques)

We recommend using the **STIX (Structured Threat Information Expression) JSON format** as the canonical source because it preserves rich markdown, includes revoked and deprecated items, and covers all domains (Enterprise, Mobile, ICS). Other formats such as Excel omit markdown and historical context, making them less suitable for fine-tuning.

2. Why Raw Data is Not Enough for Instruction Fine-Tuning

To effectively train an Instruction Fine-Tuned (IFT) Language Model, data must be structured as conversational pairs (questions and answers). The raw ATT&CK dataset is informational and structured but lacks the conversational formatting required for fine-tuning.

3. Required Format for Instruction Fine-Tuning (IFT)

Language Models expect data in a structured conversational format:

Example 1:

```
{
  "messages": [
    {"role": "system", "content": "You are a cyber-threat intelligence assistant."},
    {"role": "user", "content": "Explain what spear-phishing attachment is."},
    {"role": "assistant", "content": "Spear-phishing attachment involves adversaries sending emails with malicious attachments designed to gain initial access to systems."}
```

]
}

—----------—----------—----------—----------

Example 2:

{"messages":[

{"role":"system","content":"You are a cyber-threat assistant."},

{"role":"user","content":"List two mitigations for technique T1566.001."},

{"role":"assistant","content":"1. Use secure email gateways ... 2. Provide phishing awareness training ..."}

],

"metadata":{"technique":"T1566.001","verified":false}}

Each conversational pair must contain:

- **System prompt** (defines assistant behavior)
- **User instruction** (clear, conversational question)
- **Assistant response** (precise, accurate, complete answer)

4. Step-by-Step Procedure to Generate IFT Data

Step 1: Download and Parse MITRE ATT&CK Data

- **Download**: Clone ATT&CK STIX bundles

git clone https://github.com/mitre-attack/attack-stix-data.git

- **Example STIX Object:**


```
{
  "type": "attack-pattern",
  "id": "attack-pattern--4ab98bfb-9252-45db-a406-2268ab7b35f9",
  "created": "2024-10-01T12:15:00.000Z",
  "name": "Spear-phishing Attachment",
  "external_references": [
    {
      "source_name": "mitre-attack",
      "external_id": "T1566.001",
      "url": "https://attack.mitre.org/techniques/T1566/001/"
    }
  ],
```

```
  "description": "Adversaries send emails with malicious attachments...",
  "x_mitre_platforms": ["Windows", "macOS", "Linux"]
}
```

- **Parse**: Extract relevant fields from STIX JSON

```
import stix2, json

bundle = stix2.parse(open("enterprise-attack.json").read())
techniques = [obj for obj in bundle.objects if obj.type == "attack-pattern"]
```

Step 2: Convert Parsed Data into Question-Answer Format

- Use Python templating to generate conversational pairs:

```
ift_pairs = []
for technique in techniques:
    q = f"Explain the technique {technique.name}
({technique.external_references[0]['external_id']})."
    a = technique.description
    pair = {
        "messages": [
            {"role": "system", "content": "You are a cyber-threat intelligence assistant."},
            {"role": "user", "content": q},
            {"role": "assistant", "content": a}
        ]
    }
    ift_pairs.append(pair)
```

Step 3: Enhance Data (Optional but Recommended)

- Use an external LLM (e.g., ChatGPT) to diversify question styles.
  Example Prompt: Using the text below, generate three diverse security-analyst style questions a junior SOC analyst might ask and provide detailed answers.

- Perform LLM-assisted review to improve accuracy and naturalness.

Similar process is used in **SecKnowledge** data preparation:

1. **Start with a handful of seed examples**

The authors already have thousands of *template-made* Q-A pairs, e.g.

```
### Instruction
Map CVE-2021-34527 to its CWE weakness.

### Answer
CWE-269: Improper Privilege Management
```

2. **Feed those examples to a powerful LLM (e.g. GPT-4) as a "few-shot" prompt**

The prompt literally shows several full Q-A pairs and then asks the model to create more:

```
Below are 3 examples of cybersecurity questions with their correct answers.

1. Q: Map CVE-2021-34527 to its CWE weakness.
   A: CWE-269: Improper Privilege Management

2. Q: List two mitigations for technique T1566.001.
   A: 1. User training  2. Secure email gateway

3. Q: What ATT&CK tactic does T1190 belong to?
   A: Initial Access

Generate 5 NEW instruction-answer pairs of comparable style and difficulty.
```

3. **LLM invents new pairs**

GPT-4 might return:

```
4. Q: Which data-source log would best detect technique T1059?
   A: Process command-line logs

5. Q: Give the CVE ID exploited by WannaCry ransomware.
   A: CVE-2017-0144
...
```

4. **Post-filter**

Automatic checks: remove duplicates, ensure answer text actually appears in a trusted source, length limits, etc.

Step 4: Save in JSONL Format

- Store the generated conversational pairs efficiently:

```
with open("attack_ift_data.jsonl", "w") as f:
    for pair in ift_pairs:
        f.write(json.dumps(pair) + "\n")
```

Step 5: Tokenization for Model Training

- Use the model tokenizer (e.g., LLaMA-3) to apply conversational templates and prepare for fine-tuning:

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Meta-Llama-3-8B-Instruct")
def tokenize_pair(pair):
    return tokenizer.apply_chat_template(pair["messages"],
add_generation_prompt=False)

# Tokenize dataset
tokenized_dataset = [tokenize_pair(pair) for pair in ift_pairs]
```

Input:

```
[
  {"role":"system","content":"You are a cyber-threat intelligence
assistant."},
  {"role":"user","content":"Explain what spear-phishing attachment
is."},
  {"role":"assistant","content":"Spear-phishing attachment involves
adversaries sending emails with malicious attachments designed to
gain initial access to systems."}
]
```

Output:

```
<s>[INST] <<SYS>>
You are a cyber-threat intelligence assistant.
<</SYS>>

Explain what spear-phishing attachment is.
[/INST] Spear-phishing attachment involves adversaries sending emails
with malicious attachments designed to gain initial access to
systems.</s>
```

5. Summary

By following these steps, raw MITRE ATT&CK data (in STIX format) can be effectively transformed into high-quality conversational training data suitable for fine-tuning Athena-CTI-LLM. This structured approach ensures the resulting model can accurately and effectively perform cybersecurity-specific conversational tasks.

MISP

Overview of MISP and Its Datasets

The **Malware Information Sharing Platform (MISP)** is not just software for threat intel sharing, but also provides several structured **datasets** and formats for cyber threat intelligence . On the MISP project website, the key data resources include:

- **MISP Core Format** – a JSON schema for threat events (attributes and indicators) shared via MISP (useful for exchanging threat reports, but not directly Q&A content) .
- **MISP Taxonomies** – pre-defined classification schemes (tags) for labeling events (e.g. incident categories, threat levels). These are simple vocabularies (with short descriptions) used for tagging, rather than detailed knowledge content .
- **MISP Objects** – standardized templates to describe complex data (like a *device*, *person*, *malware sample*, etc.) as collections of attributes . They model how to share structured info but don't themselves provide explanatory text for Q&A.
- **MISP Default Feeds** – a set of open threat feeds (lists of IoCs like malicious IPs, domains, hashes) in various formats . These feeds are valuable for threat *correlation*, but since they're mostly lists of indicators (with minimal context) they aren't directly useful for generating dialogue content.

The most **useful datasets for creating instructional dialogues** are the **MISP Galaxy clusters**.

MISP Galaxy: Rich Knowledge for Q&A

MISP Galaxies are collections of *clusters*, each cluster being a curated set of related threat intelligence **entities with descriptions** . In essence, galaxies serve as a built-in knowledge base of threat actors, tools, malware, attack techniques, etc., that can be attached to MISP events. Many galaxy clusters correspond to well-known frameworks or domains in cybersecurity . For example, MISP galaxies cover:

- **Threat Actor clusters** – profiles of adversary groups (e.g. APT28, FIN7), including aliases, attribution, and background info.
- **Malware clusters** – descriptions of malware families.
- **Tool clusters** – descriptions of hacking tools or software used by adversaries.
- **Vulnerability clusters** – info on notable vulnerabilities (e.g. CVE-based clusters).
- **Framework matrices** – complete data from standards like **MITRE ATT&CK** (enterprise, mobile, ICS), **MITRE D3FEND**, **NIST attack frameworks**, etc. MISP galaxies explicitly include MITRE ATT&CK techniques (called *Attack Pattern* cluster), ATT&CK mitigations (the *Course of Action* cluster), ATT&CK threat

groups (*Intrusion Set*), ATT&CK software, and so on . They also have clusters for related domains (e.g. the **Preventive Measure** galaxy of general defenses, or DISARM framework's *Detections* and *Countermeasures*clusters).

Each galaxy cluster is provided as a JSON file (available via MISP or on GitHub) containing a list of **elements**. Each element typically has:

- a **value** (name or identifier of the item, sometimes including an ID),
- a **description** (rich explanatory text),
- and optionally a **meta** field (with extra info like references, aliases, IDs, etc.).

For instance, in the **MITRE ATT&CK "Attack Pattern"** cluster (enterprise techniques), an element might have a value like **"T1566.001 Spearphishing Attachment"** and include a description explaining that technique . Similarly, the **"Course of Action"** cluster contains MITRE's mitigation measures, e.g. an element **"M1017 User Training"** with a description about training users to recognize phishing .

How many clusters are useful? MISP comes with dozens of galaxy clusters (over 60+ listed on the site) , but not all are equally useful for Q&A. For instruction fine-tuning, we focus on clusters with **descriptive, explanatory content**:

- **MITRE D3FEND** – defensive techniques mapping; could enhance Q&A about detection or countermeasures for certain attacks.
- **General defensive measures** – e.g., the **Preventive Measure** galaxy cluster provides common security best practices (firewalls, network segmentation, etc.). This overlaps with mitigations and can supply answers for general "How can we prevent X?" questions.
- **DISARM Detections/Countermeasures** – clusters from the DISARM framework that list detection techniques and countermeasures in generic terms. These could be used for questions like "How to detect activity X?" if mapped appropriately.

On the other hand, clusters that are essentially enumerations or codes (e.g. **NACE/NAICS sector codes, NICE framework roles, sigma rules listings**) are less useful for dialogues – they contain structured lists or identifiers rather than explanatory text.

Structure of the Data and Conversion to Dialogue Format

Data Format: The galaxy cluster JSON files can be obtained by downloading from the MISP website or GitHub. (MISP lets you browse or download galaxies in JSON or PDF form , and the official GitHub repo MISP/misp-galaxy provides the raw JSON). Once downloaded, these JSON files have a structure like:

```
{
  "name": "Attack Pattern",
  "description": "ATT&CK technique cluster...",
  "type": "mitre-attack-pattern",
  "values": [
    {
      "uuid": "...",
      "value": "T1566.001 Spearphishing Attachment",
      "description": "Adversaries may send spearphishing emails with a malicious attachment in an attempt
to gain access to victim systems. Spearphishing attachment is a specific variant of spearphishing...
[oai_citation:15‡attack.mitre.org](https://attack.mitre.org/techniques/T1566/001/#:~:text=Adversaries%20
may%20send%20spearphishing%20emails,1%5D%20Spearphishing%20may%20also)",
      "meta": {
        "ATT&CK Tactic": ["Initial Access"],
        "technique_id": "T1566.001",
        "external_id": "T1566.001",
        "refs": [ "https://attack.mitre.org/techniques/T1566/001/" ],
        // ... possibly other fields
      }
    },
    {
      "value": "T1566.002 Spearphishing Link",
      "description": "...",
      "meta": { ... }
    },
    ... (many more)
  ]
}
```


Procedure – from Download to IFT-Ready Dialogues:

1. **Download/Extract Data:** First, obtain the relevant galaxy JSON files. For example, fetch the **MITRE ATT&CK technique** cluster and **mitigations** cluster. (This can be done by cloning the MISP galaxy repo or via MISP's API. The site's content is open-source and frequently updated .)
2. **Parse and Transform:** Next, write a script or use a tool to parse the JSON. We then **define question templates**that use the data. Some straightforward Q&A constructions ("pre-defined questions with zero effort") include:

   Definition questions: **"Explain what Technique Name is."** – The assistant's answer is the technique's description from the dataset. For example, for **Spearphishing Attachment (T1566.001)**, the dataset description says: *"Adversaries may send spearphishing emails with a malicious attachment in an attempt to gain access to victim systems…"* . We can simplify this into a concise answer.

   Example dialogue:

Detection questions: If detection guidance is available (e.g., the **D3FEND** or DISARM detections cluster), we could have Q&A like **"How can one detect [Technique]?"** and answer with known detection methods (for spearphishing attachment, the dataset might suggest monitoring email logs, scanning attachments, etc.). This might require combining data (ATT&CK's *Detection* section for the technique, or using the Data Source cluster).

3. **LLM-Generated Q&A (Augmentation):** Then we can augment this data using ChatGPT to generated full-dialogues.

AlienVault OTX

Data Types and Content Available

- **Indicators of Compromise (IoCs):** OTX "pulses" consist of lists of IoCs contributed by the community. Supported IoC types include IPv4/IPv6 addresses, domain names, URLs, file hashes, email addresses, hostnames, and even CVE identifiers . Each pulse links a collection of these indicators to a specific threat or campaign.

- **Threat Descriptions and Tags:** Each pulse has a title and a description field describing the threat context or attack campaign. Pulses often include **tags** (keywords for malware families, attack techniques, victim sector, etc.) and may be linked to an **adversary name** (threat actor) provided by the author . This contextual info helps explain what the indicators represent (e.g. *"IoCs related to XYZ malware used by ABC threat group"*).

- **References and Attribution:** Pulses can contain reference URLs to source material (blog posts, reports) that the contributor used to create the pulse. These references give additional context or technical detail about the threat. Pulses also record metadata like the author (contributor) and a reliability or TLP level of the data , indicating how trustworthy or sensitive the threat information is.

Feeds and Bulk Data Access

- **OTX DirectConnect API:** OTX provides a REST API (requires a free API key) to programmatically retrieve pulses and indicators. This API allows synchronization of threat data into other tools . For example, one can call endpoints to get pulses by ID, query pulses by keywords, or fetch all pulses from subscribed users. The API supports filters like date ranges (e.g. *modified_since*) to pull incremental updates of new pulses.

- **Structured Data Formats:** In addition to the API, pulses can be exported in bulk via the web portal in formats such as CSV, STIX, or OpenIOC . These formats enable downloading all IoCs and details from a pulse in a machine-readable way. For instance, a security team could schedule a daily export of new pulses in STIX format for offline processing. OTX also offers integration connectors (e.g. TAXII feeds, SIEM integrations) for continuous updates . Using these, one can **schedule regular downloads** of threat intel (e.g. every hour) to keep an up-to-date dataset of pulses and IoCs.

Suitability for IFT Q&A Generation

OTX pulse content is relatively structured and **well-suited for generating Q&A pairs** that teach a model threat intelligence analysis:

- The presence of clear fields (threat description, IoC list, CVE IDs, etc.) allows creation of straightforward questions. For example, given a pulse, one could ask: *"What indicators of compromise are listed?"*, *"Which CVE does this pulse reference?"*, or *"Summarize the threat described by this pulse."* The answers can be directly extracted from the pulse data (IoC list, CVE field, description text).

- Pulse descriptions (though often short) can be used to prompt summarization or explanation tasks. For instance, if a pulse description says a malware targeting a certain industry, a question could be: *"What is the target or focus of this threat?"* and the answer would summarize the description.

- Because pulses link multiple IoCs to a threat, they enable **list-type answers** (e.g. *"List the domain names associated with this campaign"*). This is useful for instructing a model to enumerate items from data.

- The content is generally factual and technical, which is ideal for IFT: the model can be trained to output precise, factual answers (like listing IoCs or naming a CVE) without requiring subjective or creative generation.

Overall, OTX provides **structured threat intel snippets** that can be converted into QA form to train a model on identifying key details (indicators, vulnerabilities, threat actors) from a brief report.

Example Transformation to IFT Q&A JSON

Below is an example of how an OTX pulse could be transformed into a conversational instruction-answer pair in JSON format:


```
[
  {
    "role": "system",
    "content": "You are a threat intelligence assistant with access to AlienVault OTX data."
  },
  {
    "role": "user",
    "content": "The OTX pulse indicates a Microsoft Exchange Server hack. It includes CVE-2021-26855 among its IOCs. Which CVE was exploited in this OTX pulse?"
  },
  {
    "role": "assistant",
    "content": "The pulse references CVE-2021-26855 as the exploited vulnerability in the Exchange Server attack."
  }
]
```


In this example, the **user** asks about a detail from a pulse (“Which CVE was exploited?”), and the **assistant** responds with the CVE found in that pulse’s data.

Google Project Zero

Data Types and Content Available

- **Vulnerability Research Reports:** Google’s Project Zero blog provides in-depth technical reports on discovered vulnerabilities, often **zero-day exploits** in popular software. These reports include detailed descriptions of the vulnerability, how it was found, and sometimes how it was exploited in the wild. For example, a Project Zero post might analyze a Windows kernel bug or an Android exploit chain, explaining the bug’s root cause and the exploit technique step by step . Many posts focus on memory corruption issues (buffer overflows, use-after-free, etc.), complete with code snippets and debugging details.

- **CVE References and Issue Tracking:** Posts typically reference the relevant CVE identifiers and Project Zero issue tracker numbers for the vulnerabilities discussed. Often the title or introduction will explicitly mention the CVE (e.g. *"Exploiting CVE-2022-42703"* or *"Analysis of CVE-2023-0266 in Android"*) . This makes it easy to identify which vulnerability is being discussed. They also mention when the bug was fixed (patch release info) and sometimes link to external resources like vendor advisories or commits.

- **Exploit Walkthroughs:** A unique aspect of Project Zero content is the **exploit development narratives**. The researchers often provide a walkthrough of how they turned a bug into a working exploit, including overcoming mitigations. This includes pseudocode, actual exploit code fragments, debugger output, and explanations of techniques (e.g. heap grooming, infoleaks, bypassing ASLR). For instance, a blog series on Windows registry vulnerabilities not only described the bug but also how to achieve privilege escalation with it .

- **Root Cause Analysis:** Many posts dedicate a section to root cause analysis – explaining exactly what programming flaw led to the vulnerability (e.g. *"an integer overflow in function X allows out-of-bounds write"* or *"a race condition in module Y leads to use-after-free"*). They often illustrate these root causes with snippets of the vulnerable code and discuss why existing defenses failed to catch it. The **"In the Wild"** series (where they analyze zero-days exploited in the wild) explicitly separates the root cause analysis from the exploitation discussion .

- **Context and Impact:** The blogs are rich in context: they may describe how the vulnerability was discovered (fuzzing, code audit, etc.), mention if it was exploited by attackers (and by whom if known), and the broader implications. For example, they might note if a bug was part of a larger campaign or how it compares to similar vulnerabilities. Some posts include **timelines** of disclosure and patching, or discuss defense suggestions. All these details can be mined for Q&A (e.g. *"When was this bug reported to the vendor?"* or *"What mitigation could have prevented this exploit?"*).

Feeds and Bulk Access

- **RSS/Atom Feed:** The Project Zero blog (hosted on Blogger) provides an Atom feed of its posts. This feed URL (e.g. https://googleprojectzero.blogspot.com/feeds/posts/default) can be used to programmatically retrieve the latest blog entries . By polling this feed, one can get post metadata (title, author, date) and often either a summary or the full content in XML format. This is ideal for scheduling downloads of new posts as they are published.

- **Web Scraping:** For bulk access to older posts, one may need to scrape the blog's archive. The Project Zero blog has been active for years, and posts are archived by date on the site. A scraper can navigate the blog pages or use the feed with parameters (like ?start-index or ?max-results on the feed URL) to retrieve all historical posts. Since the feed might only provide recent items, scraping the HTML of each post URL ensures no content is missed.

- **Content Format:** Each blog post is an HTML page. It typically includes the main content (which can be several pages long), code blocks, and images (like diagrams or charts). When fetching programmatically, one can use an HTTP client to download the HTML and then parse it. The **Blogger platform** tends to structure content within known tags (e.g. might contain the article). Alternatively, third-party APIs or Blogger's GData API could be used if available, but the RSS + direct fetch method is usually sufficient.

- **Update Schedule:** New Project Zero posts are irregular (whenever research is ready). To capture them, one could run a daily check of the RSS feed or subscribe to email updates and trigger the pipeline when a new post is detected. Bulk retrieval is typically a one-time effort to get all past posts, after which incremental updates via the feed suffice.

Suitability for IFT Q&A Generation

Project Zero's content is **highly suitable** for creating challenging IFT Q&A, especially for technical comprehension:

- **Detailed Explanations:** The posts contain explanations of complex concepts in security, which can be turned into explanatory QAs. For example, *"Explain in simple terms the root cause of CVE-2021-0920."* The answer would force the model to summarize a dense technical root cause into layman terms, a good exercise in comprehension .

- **Factual Questions:** Because posts reference CVEs, functions, and technical outcomes, we can ask very specific factual questions. E.g., *"What function was vulnerable to buffer overflow?"*, *"Which CVE was used in the exploit chain?"*, or *"What was the impact of exploiting this bug?"*. The answers are typically explicitly stated in the text (or can be inferred from it), which makes them reliable training data (the model can learn to locate and state these facts).

- **Step-by-Step Reasoning:** Some questions can target the exploit steps. For instance, after a post explains Steps 1, 2, 3 of an exploit, we can query: *"What was the first step the researchers took to exploit the vulnerability?"* The answer could be one of the steps (e.g. *"They used an information leak to bypass ASLR in*

Step 1."). This trains the model on procedural understanding.

- **Summarization and Synthesis:** The richness of the content allows for higher-level tasks like summarization: *"Summarize the blog post about the Android exploit chain in one paragraph."* or *"What are the key takeaways from this Project Zero report?"* The model needs to condense the complex report into a coherent summary, which is a valuable skill for an AI assistant.

- **Complex Q&A:** One can also generate multi-hop questions that require combining info from the post. For example: *"According to the Project Zero analysis, how was CVE-2023-0266 initially discovered and what makes it significant?"* – the answer would involve stating the discovery method and the significance, which might be found in different parts of the text. This mimics a user asking an expert to interpret the report. Such Q&A pairs teach the model to integrate information and provide a concise explanation.

Example Transformation to IFT Q&A JSON

Here is an example of a Q&A pair derived from a Project Zero blog post, formatted for an instruction-tuned model:


```
[
  {
    "role": "system",
    "content": "You are a cybersecurity expert specializing in software vulnerabilities."
  },
  {
    "role": "user",
    "content": "The Project Zero blog describes CVE-2022-34169 as an integer truncation bug in a Java JIT compiler leading to code execution [oai_citation:17‡wilderssecurity.com](https://www.wilderssecurity.com/threads/news-and-updates-from-the-project-zero-team-at-google.427577/page-3#:~:text=,on%20standard). Explain the root cause of this 0-day in simple terms."
  },
  {
    "role": "assistant",
    "content": "Sure. CVE-2022-34169 was caused by an integer value getting cut off (truncated) because it was too large for its data type. This truncation happened in Java's JIT compiler during XML signature verification, and it led to a miscalculation in memory size. Because of that mistake, the program could write data to the wrong place in memory, which an attacker could exploit to execute code."
  }
```

]

In this example, the **user** prompt references details from the blog (the CVE and its nature) and asks for a simple explanation of the root cause. The **assistant** answer is a concise rephrase of the technical root cause in more general terms, suitable for a broad audience. We even included a brief citation in the user prompt for clarity (in practice, the actual prompt might include an excerpt of the blog as context).

Another example could be a direct question: *"Which CVE from the Project Zero 'In-the-Wild' 2023 report was a 0-day in the ALSA driver?"* and the assistant would answer with the CVE (e.g., *"CVE-2023-0266"*). By providing these Q&A examples to the model during fine-tuning, we teach it how to answer questions based on detailed vulnerability reports.

Parsing and Extracting Project Zero Data

Handling Project Zero blog content requires robust parsing and cleaning, given the posts' length and formatting:

- **HTML Parsing:** Fetch the blog post HTML and parse it (using libraries like BeautifulSoup). The goal is to extract the main content text. Typically, one would target the container that holds the post body (for Blogger, it might be a). Strip out navigation menus, author bio, comments, or any unrelated text. The parser should also capture text in code blocks or blockquotes if needed (though for Q&A we might exclude large code dumps unless questions about specific code are desired).

- **Handling Code and Images:** Many Project Zero posts contain code snippets and images (diagrams). Decide on a policy for these:

    - For code: If the goal is Q&A about the code (e.g. *"What does this snippet do?"*), preserve it. Otherwise, you might remove or summarize code sections to focus on prose. A compromise is to keep function names or key lines as plain text since the narrative often refers to them.

    - For images: They usually illustrate memory layouts or exploit flows. Since we can't directly use images in text Q&A, ensure any crucial information from captions or image descriptions is retained. Often the text around the image explains what the diagram shows , so the parser should include those explanations.

- **Text Cleaning:** Project Zero posts are well-written but can be very lengthy. Clean the extracted text by removing excessive newlines, weird Unicode characters (if any), and any references to figure numbers or internal anchors that lost meaning outside HTML. It's also wise to preserve section headings from the blog as they indicate structure (e.g. "Background", "Exploitation", "Conclusion"). These can later help in splitting the content or focusing question generation on specific sections.

- **Metadata extraction:** From the post, we can scrape the title, publication date, and author. The title often contains the vulnerability name or theme (which can be used in questions, e.g. *"In [Title], how did the exploit bypass ASLR?"*). The date might be useful for timeline questions (e.g. *"When was this analysis published?"*). Author is less critical for technical Q&A, but could be used if we want to credit or identify perspective (e.g., *"According to Jann Horn's findings, …?"*).

- **Segmentation:** Given the size of some posts, it can help to segment the content by logical sections before generating Q&As. For example, separate the root cause analysis part from the exploitation part. This way, questions can be generated from each segment without the LLM or template having to consider the entire lengthy text at once. We might store these segments in a structured way (like an array of sections).

- **LLM-assisted extraction:** Project Zero content can be very dense. An LLM can assist by summarizing sections or extracting key points:

    - Summarize each section into a few bullet points (using an LLM) to identify potential question material.

    - Ask the LLM questions about the text to see if it correctly identifies the answers (as a form of validation). For instance, after parsing, we could prompt, *"What are the vulnerabilities mentioned in this post?"* and see if it answers with the CVEs – verifying we didn't miss any during parsing.

    - Use the LLM to **simplify complex language**. For training an assistant, we may want understandable answers. If the blog uses very technical jargon, we might run the answer through a simplification prompt (or have the model generate the answer directly in simpler terms, as part of Q&A generation).

Careful parsing ensures we have the complete textual content of each blog post in a usable form. This content is the foundation from which we will generate Q&A pairs.

CVE/ NVD (National Vulnerability Database)

Overview of NVD and Its Vulnerability Datasets

The **National Vulnerability Database (NVD)**, maintained by NIST at https://nvd.nist.gov, is the U.S. government's authoritative source for vulnerability management data. Built on the CVE system and powered by the Security Content Automation Protocol (SCAP), the NVD provides rich, structured data including:

- **CVE identifiers**
- **CVSS scores** (severity metrics)
- **CWE classifications** (weakness types)
- **CPEs** (affected software/hardware platforms)

The database supports automation through JSON/XML data feeds and a robust version 2.0 API, enabling integration into real-time threat management, compliance, and security tools. This report explores how NVD data can be converted into an Instruction Fine-Tuning (IFT) dialogue format to train models for cybersecurity tasks.

Structure of NVD Vulnerability Records

Each NVD CVE entry includes:

- **CVE ID and Description**
    - Example: *CVE-2025-24813* – A path-equivalence vulnerability in Apache Tomcat (v9.0.x to 11.0.2) allowing potential RCE or content injection via filename manipulation (file.Name).
- **Publication & Modification Dates**
    - Published: *March 3, 2025*
    - Last Modified: *April 3, 2025*

- **Severity Scoring (CVSS)**
    - CVSS v3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
    - Score: **9.8 (Critical)**
- **Common Weakness Enumeration (CWE)**
    - Includes references like **CWE-44** or **CWE-502**, subject to revision via the NVD's CVE change log.
- **Affected Products (CPEs)**
    - Structured under cpe23Uri, identifying impacted versions such as Tomcat 9.0.0.M1–11.0.2.
- **References**
    - Vendor advisories, Apache mailing lists, public POCs, and documentation links.
- **Change History**
    - Tracked via the CVE History API for transparency on score updates, CWE mappings, and metadata corrections.

This standardized data schema supports direct extraction and mapping into conversational formats.

NVD Data Access Formats

- **JSON Data Feeds**:
    - Available by year (e.g., nvdcve-1.1-2025.json), and in delta modes (recent, modified), updated every 2 hours.
- **REST API v2.0**:
    - Provides endpoints for /cves/, /cpes/, and /cpematch/ with support for filtered queries, pagination, and real-time sync.

- **Search Interface**:
    - Web search via CVE ID, vendor, product, or OVAL definitions.

These machine-readable formats enable bulk ingestion, parsing, and dialogue generation pipelines.

Converting NVD Records to Instructional Dialogue (IFT Format)

To train instruction-following models, NVD content can be transformed into the **Instruction–Input–Output** format required for Instruction Fine-Tuning (IFT):

Format Mapping

Component	Mapped Field(s)
Instruction	User query (e.g., “Summarize CVE-2025-24813”)
Input	Contextual identifier (CVE ID, product)
Output	Summary with description, severity, products, and mitigations

Example

- **Instruction**:
  “Summarize the vulnerability for CVE-2025-24813.”

- **Input**:
  CVE-2025-24813

- **Output**:
  CVE-2025-24813 is a path-equivalence vulnerability in Apache Tomcat 9.0.0.M1 to 11.0.2, allowing attackers to bypass servlet security and upload malicious files. If file write is enabled, this may lead to remote code execution. Severity is rated

9.8 (Critical). Affected deployments should upgrade to version 11.0.3 or later.

Conversion Process

To scale NVD-to-IFT conversion:

1. **Data Extraction**
    - Use NVD APIs (/cves/, /cpematch/) or JSON feeds (nvdcve-1.1-recent.json) to extract CVE metadata.
2. **Dialogue Generation**
    - Map CVE descriptions to the output field. Use CVE ID or affected product as input. Template the instruction accordingly.
3. **Web and API Integration**
    - NVD data can be accessed either via its official REST API or by programmatically scraping structured web content. The API is ideal for scalable, rule-based queries (e.g., by CVE ID, keyword, or CPE name), while web scraping can be used to extract additional contextual details such as formatted advisories, usage notes, or UI-specific metadata not exposed in the API. This hybrid access enables comprehensive coverage and flexibility for dataset builders.
4. **Update Scheduling**
    - Pull recent or modified feeds every 2 hours to ensure up-to-date dialogue data.

The NVD provides rich, structured data ideal for instruction-based dialogue generation. By transforming CVE records into instruction-input-output triplets using standardized templates and automated pipelines, one can create high-quality datasets for training cybersecurity-aware LLMs. Addressing challenges like accuracy, volume, and simplification ensures the reliability of these models for real-world applications.

Exploit Database (Exploit-DB)

The **Exploit Database (Exploit-DB)**, operated by Offensive Security, is a comprehensive, CVE-aligned archive of exploits, shellcodes, and PoCs aimed at penetration testers and researchers. With over **46,000 files**, it offers practical code and metadata that greatly complement static vulnerability records, enabling educational and contextual exploitation learning. [github.com+12exploit-db.com+12github.com+12gitlab.com+5en.wikipedia.org+5github.com+5]

Structure of Exploit-DB Entries

Each exploit entry includes:

- **EDB-ID & CVE**
  Unique identifiers (e.g., *EDB-ID: 42599*) and CVE links when available. exploit-db.com+3exploit-db.com+3exploit-db.com+3

- **Title & Type**
  Descriptive title (e.g., "Git <2.7.5 – Command Injection") and class (`remote`, `local`, `webapps`, `shellcode`, etc.). exploit-db.com+6exploit-db.com+6exploit-db.com+6

- **Author & Verification**
  Contributor info (e.g., *Metasploit*) plus verification status.

- **Platform & Date**
  Target language/platform (e.g., Python), and publication date (e.g., *2017-08-31*). exploit-db.com+1exploit-db.com+1

- **Exploit Metadata & Comments**
  Embedded comments indicate vendor, version, test environment, or advisory links.

- **Exploit Code**
  Full PoC script—Python, Ruby, C, Metasploit modules—available inline or from the repository.

- **External References**
  Links to advisories, mailing lists, GitHub, etc.

Exploit-DB Data Access Methods

Exploit-DB supports three primary access methods:

1. **Git Repository / `SearchSploit`**
   - Clone the repo from GitLab: `git clone https://gitlab.com/exploit-database/exploitdb.git` cliffsnotes.com+15exploit-db.com+15github.com+15
   - Use `SearchSploit` for local search and retrieval: `searchsploit --cve CVE-xxxx-yyyy` returns exploit path and metadata. youtube.comexploit-db.com+5exploit-db.com+5kali.org+5

2. **Web Scraping**
    - Exploit pages contain structured HTML fields. With 46,000+ files and included web-scraping PoCs, custom scrapers can extract metadata and full exploit content at scale.
3. **API and Community Tools**
    - Third-party utilities like `pyxploitdb` or `go-exploitdb` fetch and store data into local SQLite, supporting automated pipelines. github.com

These methods support both bulk ingestion and fine-grained retrieval workflows.

Converting Exploit-DB Data to IFT Dialogue

Generate **Instruction–Input–Output** triplets as follows:

Field	Mapping Example
Instruction	“Provide exploit code for CVE-2017-1000117.”
Input	`CVE-2017-1000117`
Output	Summary and PoC code with usage, e.g., Python Metasploit module. exploit-db.com+1exploit-db.com+1

Example:

- **Instruction:**
  "Provide the exploit for CVE-2017-1000117."

- **Input:**
  CVE-2017-1000117

- **Output:**
  The Ruby Metasploit module targeting Git <2.7.5, including usage details and exploit code snippet. exploit-db.com+1exploit-db.com+1

Conversion Workflow & Data Integration

1. **Repository Syncing**
    - Use `git clone` and schedule `git pull` daily.
    - Keep replicate of both `exploitdb` and `exploitdb-bin-sploits` for binaries. exploit-db.comexploit-db.com+14exploit-db.com+14exploit-db.com+14
2. **Search & Extract**
    - Query with `SearchSploit` (e.g., `--path`, `--json`).
    - Locate exploit files under `/exploits/` or `/shellcodes/`, then extract metadata and code.
3. **Web Scraping Augmentation**
    - Use PoC scraping scripts found in the repository to automatically extract from HTML entry pages, capturing details not present in git.
4. **Dialogue Generation**
    - Parse metadata and PoCs; apply templates to build IFT triplets combining summary and code.
    - Support variants: code-only, explanation-only, or combined.
5. **Update Automation**
    - Use cron or CI pipelines to:
        - `searchsploit -u`
        - `git pull`
        - Run scraper
        - Regenerate dialogues

Exploit-DB's extensive archive, combined with repository access and automated scraping, enables scalable creation of instruction-tuned cybersecurity dialogue datasets. By pairing exploit metadata with PoC code in structured IFT format, you can train models capable of delivering both technical details and actionable context—provided that robust quality controls and ethical safeguards are in place.

Rapid7 Github

Rapid7 GitHub – Data-Focused Repositories and How to Turn Them into IFT Dialogue Corpora

Rapid7 hosts several repositories whose **primary value is structured threat-intelligence data, not executable code**. These collections—fingerprints, default SSH

keys, and Internet-wide scan artefacts—can be transformed into high-quality *Instruction · Input · Output* triples for instruction-fine-tuning (IFT) of security-aware language models.

1 · Recog – protocol-fingerprint library

Repository: `rapid7/recog`
XML files under `recog/xml/` map banners, headers and TLS traits to vendor, product, OS, and version. Each fingerprint carries a short description plus one or more verified `<example>` strings—perfect "ground truth" for Q-and-A generation.
Acquire: `git clone https://github.com/rapid7/recog.git` (≈ 35 MB).

Typical IFT triple

- *Instruction* Identify the operating system from this SSH banner.
- *Input* `SSH-2.0-OpenSSH_9.6 FreeBSD-20250620`
- *Output* FreeBSD 13.x running OpenSSH 9.6 (matched via *ssh_banners.xml*).

2 · SSH-BadKeys – default key blacklist

Repository: `rapid7/ssh-badkeys`
Lists of public and private keys that ship in commercial products, grouped by key type and accompanied by SHA-256 fingerprints.
Acquire: `git clone https://github.com/rapid7/ssh-badkeys.git` (≈ 6 MB).

IFT use-case: *"Is this RSA fingerprint black-listed?"* → "Yes—present in *badkeys-rsa-public.list* and linked to a specific IoT camera firmware."

3 · Project Sonar Open Data – Internet-wide scan results

Repository stub: `rapid7/sonar` (redirects to the Open-Data portal).
Daily DNS, HTTP, TLS, SMB and other scans are released as compressed CSV/JSON.
Acquire: list with `aws s3 ls s3://rapid7-open-data/` then pull only the files you need.

Example IFT task: *"How many IPv4 hosts were listening on port 445 on 1 June 2025?"* → numerical answer derived from that day's `sonar.tcp445` file.

4 · Rapid7/Data – research snapshots

Repository: `rapid7/data` (archived).
Contains small CSVs and notebooks that back Rapid7 blog posts and annual reports (e.g., *top-malware-2024.csv*).
IFT prompt: *"Which malware family dominated Rapid7's 2024 telemetry?"* → "QBot, ~23 % of detections."

Summary of Data Repositories

Repo	Main Content	File Types	Example IFT Task
`recog`	Protocol fingerprints	XML	Banner → product & OS
`ssh-badkeys`	Static SSH keys	PEM / hash lists	Key reputation check
`sonar`	Internet scan artefacts	gz-CSV / JSON	Exposure statistics
`data`	Research datasets	CSV	Trend summarisation

End-to-End Conversion Workflow for IFT

1. **Acquire the Data**
   • `git clone` for `recog`, `ssh-badkeys`, `data`.
   • `aws s3 cp` or similar for the required Sonar scan partitions.

2. **Pre-process & Normalise**
   • Parse XML with XPath to pull `<description>`, `<pattern>` and `<example>` from Recog.
   • Read PEM files, calculate fingerprints, and load hash lists for BadKeys.
   • Decompress Sonar CSV/JSON and filter to the fields relevant to your question templates.

3. **Design Prompts**
   Craft domain-realistic instructions that analysts actually ask:
   • “Identify the product behind this banner.”
   • “Is this key known-bad?”
   • “How many hosts exposed service X on date Y?”
   Each prompt becomes the *Instruction*; a single data artefact (banner, fingerprint, dataset identifier) becomes the *Input*.

4. **Generate Answers**
   Derive the *Output* automatically:
   • Look up the banner in Recog and return the matched vendor/product line.
   • Search the fingerprint list and state whether it is present, citing the list name.
   • Run an aggregation query over the Sonar file and return the count or a short host list.

5. **Serialize to JSONL**
   Store each trio as a one-line JSON object:
   `{"instruction": "...", "input": "...", "output": "..."}`

6. **Validate & Redact**
   • Spot-check random samples for accuracy.
   • Strip or hash any private keys or sensitive host data.
   • Attach a usage disclaimer if raw indicators are included.

7. **Automate Updates**
   Schedule a daily or weekly job that:
   • `git pull` or re-syncs S3,
   • regenerates only the triples for new or changed artefacts,
   • appends them to your master training file.

All four repositories pair *structured, machine-readable indicators* with enough human context to produce grounded, fact-checked answers. By following the workflow above you can turn banners, key fingerprints and Internet-scan slices into thousands of concise, evidence-based dialogue examples—precisely the material a large language model needs to deliver reliable, security-focused responses.

Licensing

Dataset (unique)	Used in	License / Terms	Commercial use?*
MITRE ATT&CK (all domains)	Both	CC BY 4.0 Intern'l	**Yes**
CVE / NVD	Both	MITRE Terms of Use (free re-use w/ attribution)	**Yes**
CWE	SecKnowledge	MITRE Terms of Use	**Yes**
CAPEC	SecKnowledge	MITRE Terms of Use	**Yes**
CVSS v3 spec	SecKnowledge	FIRST CC BY-SA 4.0	**Yes (share-alike)**
Sigma rule set	SecKnowledge	MIT License	**Yes**
Security StackExchange dump	SecKnowledge	CC BY-SA 4.0	**Yes (share-alike)**
ATT&CK ⇄ Data-Source mappings / CAR	SecKnowledge	CC BY 4.0	**Yes**
MISP Galaxies (D3FEND, Preventive-Measure, DISARM, etc.)	IFT	CC BY-SA 4.0	**Yes (share-alike)**

AlienVault OTX pulses	IFT	CC BY-SA 3.0	**Yes (share-alike)**
Google Project Zero blog	IFT	© Google LLC (blog content, no open licence)	**No** (fair-use excerpts only)
Trend Micro ZDI advisories / blog	IFT	© Trend Micro (no open licence)	**No** (fair-use excerpts only)
SecKnowledge dataset	SecKnowledge	CC BY-NC 4.0 (non-commercial)	**No**

*“Yes (share-alike)” means commercial use is permitted provided you attribute and license derivatives under the same CC BY-SA terms.

“No (fair-use excerpts only)” means:

- The site’s text is fully copyrighted and **does not grant a licence for commercial redistribution or bulk reuse**.
- You may quote **small, necessary snippets** (e.g., one or two sentences) for commentary, analysis, or transformation under a *fair-use* (or similar “fair-dealing”) doctrine—provided that:
    - the excerpt is *brief*,
    - it is *transformative* (e.g., used to create your own summary or answer, not simply republished), and
    - you include proper attribution.
- You **cannot** copy whole articles or large blocks of text into your training dataset or ship them in a commercial product.

Summary

Datasets	Data Formats	Download / Access Method	Key Content / Useful Datasets	Typical LLM Preparation Steps
MITRE ATT&CK	STIX 2.1 JSON (preferred), Excel	`git clone https://github.com/mitre-attack/attack-stix-data` + scheduled `git pull`	Enterprise, Mobile & ICS tactics / techniques, mitigations, relationships	Parse *attack-pattern* & COA objects → template definition, detection & mitigation Q-A pairs
MISP Galaxies	Cluster JSON files	`git clone https://github.com/MISP/misp-galaxy` or MISP API export	ATT&CK clusters, threat actors, malware, defensive measures	Extract `value` + `description`; build prompts like “Explain technique X”, “List mitigations for Y”
AlienVault OTX	JSON (REST API), CSV/STIX exports	`OTX DirectConnect API (key)`	Pulses with IoCs, CVEs, threat descriptions, tags	Map pulse fields to Q-A: list IoCs, identify CVE, summarise threat

Google Project Zero	HTML pages, Atom RSS	Poll RSS feed; `scrape blog archive for full posts`	Deep-dive vulnerability reports, CVE analyses, exploit walkthroughs	Segment by section; generate root-cause, timeline, mitigation, summarisation Q-A
NVD / CVE	JSON 1.1 feeds, REST API v2	Hourly download of *recent/modified* feeds or `/cves/` API queries	CVE IDs, CVSS, CWE, CPE, references, change history	Template “Summarise CVE-XXXX-YYYY”, “List affected products” using feed fields
Exploit-DB	PoC code files (py, c, rb), HTML metadata	`git clone https://gitlab.com/exploit-database/exploitdb.git` / `searchsploit`, plus optional web scraping	46 k+ exploits & shellcodes with EDB-ID, CVE, platform	Extract title, code path, summary; Q-A like “Provide exploit for CVE-…” or explanation-only pairs
Rapid7 GitHub (recog, ssh-badkeys, sonar, data)	XML, PEM/txt lists, gz-CSV/JSON, CSV	`git clone` individual repos; `aws s3` sync for Sonar open-data	Protocol fingerprints, default SSH keys, Internet-wide scan artefacts, research CSVs	Fingerprint → product Q-A, key reputation checks, exposure statistics queries